Docker — Complete Theory Guide

A theory-only guide to understanding Docker and its core concepts.

1. What Is Docker?

Docker is a platform used to package, distribute, and run applications in isolated environments called containers. A container contains the application and the software dependencies required to run it. This makes the application environment more consistent across different computers and servers.

2. Why Was Docker Created?

Before containers became popular, developers often faced the problem that an application worked on one computer but failed on another. Differences in operating systems, programming language versions, libraries, and configurations could cause these problems. Docker helps create a consistent environment so that an application behaves more predictably across development, testing, and production.

3. Virtual Machines vs Containers

A virtual machine emulates an entire computer and includes a complete guest operating system. Containers do not normally include a complete operating system. Instead, they share the host operating system kernel while keeping applications and their environments isolated. Because containers share the kernel, they are generally lighter and faster to start than virtual machines.

4. Docker Architecture

Docker uses a client-server architecture. The Docker client is the interface through which users or tools communicate with Docker. The Docker daemon is the background service responsible for building images, creating containers, managing networks, and managing storage. The client sends requests to the daemon, and the daemon performs the required operations.

5. Docker Engine

Docker Engine is the core technology that allows Docker containers to be created and managed. It includes the Docker daemon, the Docker API, and the Docker command-line client. The engine is responsible for managing the lifecycle of containers and the resources they use.

6. Docker Images

A Docker image is a read-only template used to create containers. It contains the application code, required libraries, dependencies, configuration files, and other components needed by the application. An image is not itself a running application. It becomes a running environment when a container is created from it.

7. Docker Containers

A container is a running instance of a Docker image. Containers provide process-level isolation, meaning that applications inside one container are separated from applications in other containers. Containers are designed to be lightweight and portable. Multiple containers can be created from the same image.

8. Image and Container Relationship

An image can be considered a blueprint, while a container is a running instance created from that blueprint. One image can be used to create many containers. Each container has its own runtime state even though the containers may have been created from the same image.

9. Dockerfile

A Dockerfile is a definition of how a Docker image should be created. It describes the base environment, application files, dependencies, configuration, and the process that should start when a container runs. The Dockerfile makes the image-building process repeatable and reproducible.

10. Docker Image Layers

Docker images are constructed from layers. Each change in the image-building process can create a new layer. Layers can be reused between images, which helps reduce storage requirements and can make image creation more efficient. The final image is formed by combining these layers.

11. Container Lifecycle

A container has a lifecycle. It is created from an image, started to execute its main process, may continue running while that process is active, and can eventually be stopped or removed. A stopped container may still exist as an object until it is explicitly removed.

12. Container Isolation

Containers isolate processes, filesystems, networks, and other resources. This isolation allows multiple applications to run on the same host without directly interfering with one another. Container isolation is an important foundation for microservices and modern application deployment.

13. Port Mapping

Applications running inside containers may listen on ports that are isolated from the host machine. Port mapping creates a connection between a port on the host and a port inside the container. This allows external clients to communicate with an application running inside a container.

14. Docker Volumes

Containers are often treated as temporary environments. Data stored only inside a container can disappear when the container is removed. Docker volumes provide persistent storage that exists independently of a container's lifecycle. Volumes are especially important for databases and other stateful applications.

15. Docker Bind Mounts

A bind mount connects a directory or file on the host machine to a location inside a container. This allows the container to access host files directly. Bind mounts are commonly useful during development because changes made on the host can be reflected inside the container.

16. Docker Networking

Docker networking allows containers to communicate with each other and with external systems. Containers can be connected to virtual networks. When multiple services belong to the same Docker network, they can communicate through that network using their service or container identities rather than relying only on the host machine.

17. Docker Compose

Docker Compose is a tool for defining and managing applications composed of multiple services. A typical backend system may contain an API service, a database, a cache, and message-processing services. Compose allows the relationships between these services to be described as one application environment.

18. Docker Registries

A Docker registry is a storage and distribution system for Docker images. Developers and organizations can push images to a registry and later pull those images onto other machines or servers. Registries may be public or private.

19. Docker Hub

Docker Hub is a public registry containing many commonly used images. It also allows users and organizations to store and distribute their own images. Private registries can be used when images should not be publicly accessible.

20. Docker and Microservices

Docker is commonly used with microservices because each service can run in its own container. Each container can have its own dependencies and can be developed, deployed, scaled, and updated independently. This makes containers a useful technology for distributed applications.

21. Docker in Development

In development, Docker helps ensure that developers use consistent versions of databases, caches, message brokers, and application runtimes. This reduces differences between developer machines and makes it easier to reproduce environments.

22. Docker in Production

In production, containers provide a standardized unit for deploying applications. A built image can be tested and then deployed to servers or cloud platforms. Container platforms can manage container placement, scaling, networking, and availability.

23. Docker Security

Docker security involves controlling what containers can access and what privileges they have. Important concepts include limiting container privileges, protecting secrets, using trusted images, minimizing unnecessary software, and keeping the Docker environment updated.

24. Docker vs Kubernetes

Docker is primarily a technology for building and running containers. Kubernetes is a container orchestration platform that manages containers across systems. Kubernetes can handle tasks such as scheduling, scaling, service discovery, and maintaining the desired state of applications. Docker and Kubernetes are related but serve different purposes.

25. The Big Picture

The general Docker flow is conceptually simple: application source code and dependencies are packaged into an image; a container is created from the image; the container runs the application in an isolated environment; persistent data is stored separately when necessary; and multiple containers can communicate through networks and work together as a complete application.